# Project Documentation

## Title: AI Fit Track API

The AI FitTrack API is used to connect the frontend and backend of the fitness tracking application. It helps users register, log in, and manage their fitness activities securely. The API uses Node.js and Express.js to handle requests and MongoDB to store user information and workout details. It also uses JWT authentication to protect user accounts and ensure secure access to the application. The main purpose of the AI FitTrack API is to manage user data, track workouts, and support personalized fitness recommendations.

**Team ID :** SWTID-2026-8651
**Team Size :** 4

**Team Leader :** PRAVEENA K
**Mail ID :** praveenakofficial1703@gmail.com

**Team Member :** PREETHI S
**Mail ID :** manikavin283@gmail.com

**Team Member :** PRINCY R
**Mail ID :** akshayaprincy94@gmail.com

**Team Member :** PRIYADHARSHINI G
**Mail ID :** priyadharshini2007ge@gmail.com

## FitSense - Personalized fitness recommendations powered by AI.

# Description

The **FitTrack AI** backend is a secure, scalable, and AI-powered RESTful application developed to simplify workout tracking and provide intelligent fitness recommendations. Built using **Node.js**, **Express.js**, **MongoDB**, and **Mongoose**, the backend exposes REST APIs that enable secure user authentication, workout management, workout searching, and AI-assisted fitness guidance.

The application follows the **Model-View-Controller (MVC)** architecture, making the project modular, maintainable, and scalable. User authentication is implemented using **JSON Web Tokens (JWT)**, while passwords are securely encrypted using **bcrypt.js** before storage in MongoDB. Every protected API validates the JWT token through middleware before allowing access to sensitive resources.

One of the key features of the project is its integration with **Google Gemini AI**, which provides personalized workout recommendations based on a user's age, fitness goals, and workout experience. The AI module also generates fitness insights by analyzing workout statistics such as total workouts completed, average workout duration, and calories burned.

The backend returns standardized JSON responses, making it suitable for integration with web, mobile, and third-party applications. Proper error handling, input validation, route protection, and modular service implementation ensure the application is reliable, secure, and easy to extend.

# Scenario-Based Case Study

### Background

Rahul is a fitness enthusiast who exercises regularly but records his workouts manually using notebooks and spreadsheets. Although he maintains consistency in his fitness routine, he finds it difficult to organize workout history, monitor progress, and determine whether his exercise habits align with his fitness goals. He also lacks professional guidance when deciding which workouts are best suited for his current fitness level.

### Problem

- Difficulty maintaining a centralized record of workouts.
- Manual tracking leads to inaccurate workout history.
- No secure user authentication for personal fitness records.
- Unable to search previous workouts efficiently.
- No personalized workout recommendations.
- No insights regarding overall workout performance.
- Time-consuming manual calculation of calories burned and workout duration.

### Solution

The **FitTrack AI** provides a centralized backend system where users can securely register, authenticate, and manage workout records. Users can add, update, delete, retrieve, and search workouts through RESTful APIs. By integrating **Google Gemini AI**, the application analyzes user information and workout statistics to generate intelligent workout recommendations and personalized fitness insights.

The backend follows industry-standard security practices including JWT authentication, password hashing, middleware-based authorization, centralized error handling, and MongoDB schema validation.

# Usage

## User Usage

A registered user can create an account using the registration endpoint and securely log in using email and password. After successful authentication, the backend generates a JWT token that authorizes access to protected APIs.

Using authenticated APIs, users can:

- Create new workout records.
- View all previously completed workouts.
- Retrieve a workout by its unique identifier.
- Update existing workout information.
- Delete unwanted workout records.
- Search workouts using workout name, category, or workout date.
- Generate AI-powered workout recommendations.
- Obtain AI-generated fitness insights based on workout statistics.

### AI Usage

The AI module communicates with Google Gemini using a dedicated service layer.

Users provide information such as:

- Age
- Fitness Goal
- Experience Level

Gemini processes this information and returns:

- Personalized workout plan
- Weekly exercise suggestions
- Beginner/Intermediate/Advanced recommendations
- Motivational guidance
- Safety tips

Similarly, users can submit workout statistics including:

- Total Workouts
- Average Workout Duration
- Total Calories Burned

The AI module analyzes these values and produces personalized fitness insights.

# Outcome

The AI FitTrack backend significantly improves fitness tracking by eliminating manual record keeping and providing intelligent recommendations based on user activity. Secure authentication ensures that only authorized users can access personal workout data, while AI assistance helps users improve their exercise routines with personalized suggestions.

The modular MVC architecture enables easy maintenance and future scalability. The application can later be extended with additional features such as nutrition tracking, BMI calculation, wearable device integration, workout reminders, and fitness analytics dashboards without major architectural changes.

### 1. Software Requirements

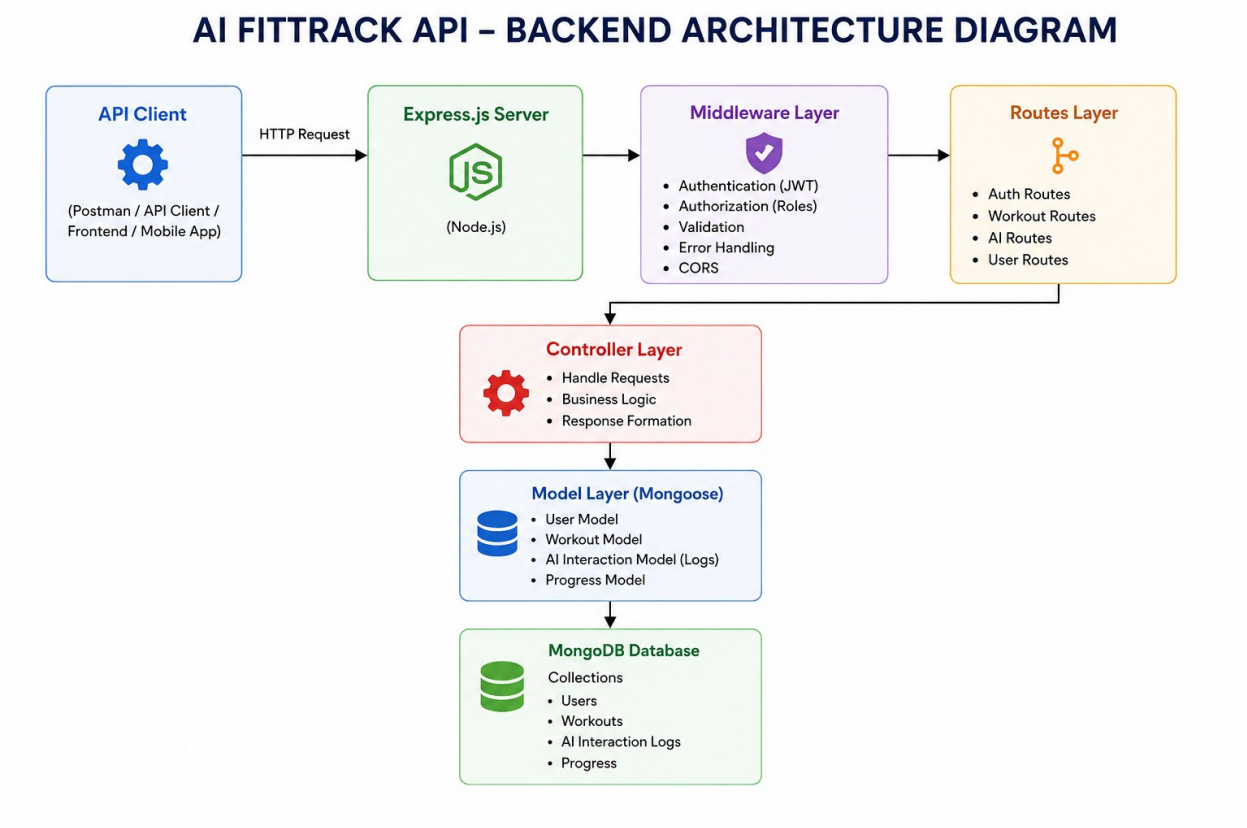
The FitTrack AI  requires the following software components for development and execution.**Operating System:** Windows 10/11, macOS, or Linux (supports cross-platform operations).

- **Node.js (v16 or above):** Runtime ecosystem running server logic and routing infrastructure.
- **npm (v8 or above):** Node package manager to control operational dependencies.
- **Express.js:** Lightweight routing web framework to construct backend RESTful entry points.
- **MongoDB:** Document NoSQL engine storing users, posts, categories, comments, and analytics metrics.
- **Postman:** API verification toolkit to assert schema validation responses across administrative route shields.
- **Code Editor:** Visual Studio Code or similar IDE environment.

### 2. Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- **RAM:** 8 GB minimum (16 GB recommended for concurrent instances of MongoDB, processing layers, and execution nodes).
- **Storage:** 1 GB of available disk workspace.

# Epic 1: Project Architecture




The FitTrack AI  follows a layered architecture based on the MVC (Model–View–Controller) design pattern. The application separates routing, business logic, database interaction, middleware, and AI services into independent modules, making the project easier to maintain and scale.

When a client sends an HTTP request, the Express server receives it and forwards it to the corresponding route. Protected routes first pass through the authentication middleware, which validates the JWT token before granting access. Controllers process validated requests and invoke business logic or AI services as required. Database interactions are handled through Mongoose models, while Google Gemini AI is accessed through a dedicated service layer. Finally, the controller returns a structured JSON response to the client.

This layered approach improves modularity, security, and maintainability while ensuring that each component has a single responsibility.

## Overall Architecture Components

- Client Application (React / Mobile App / Postman)
- Express Server
- API Routes
- Authentication Middleware
- Controllers

- Services
- Google Gemini AI Service
- MongoDB Database
- Mongoose Models

Each component communicates through well-defined interfaces, ensuring loose coupling and easier future enhancements.

# Components Definition

The FitTrack AI  follows a modular backend architecture in which every component performs a dedicated responsibility. The separation of concerns ensures maintainability, scalability, and efficient execution of requests. Each request passes through multiple layers before reaching the database or AI service, ensuring secure and reliable processing.

## 1. Client Layer (Frontend/API Consumer)

The Client Layer is the entry point of the application. It can be a React application, mobile application, or API testing tool such as Postman. The client sends HTTP requests to the backend server and receives JSON responses.

### Responsibilities

- Sends API requests.
- Displays workout information.
- Stores JWT token after login.
- Invokes AI recommendation APIs.
- Displays success and error responses.

### APIs Accessed

- Authentication APIs
- Workout APIs
- AI Recommendation APIs
- AI Fitness Insight APIs

## 2. Express Server Gateway

The Express server acts as the central gateway of the application. It listens for incoming requests, configures middleware, parses request bodies, enables CORS, and routes requests to the appropriate controllers.

### Responsibilities

- Starts HTTP server
- Parses JSON requests
- Enables Cross-Origin Resource Sharing (CORS)
- Registers application routes
- Connects middleware
- Handles centralized errors

### Example Responsibilities

- Receive HTTP Request
- Validate Request Format
- Forward Request to Routes
- Return JSON Response

## 3. Authentication Middleware

Authentication middleware protects all secured APIs by validating the JSON Web Token (JWT). Every protected request must include a valid Authorization header.

### Responsibilities

- Read Authorization Header
- Extract JWT Token
- Verify JWT Signature
- Decode User Information
- Attach User Information to Request
- Reject Unauthorized Requests

### Benefits

- Prevents unauthorized access
- Secures workout information
- Allows only authenticated users to access protected APIs
- Eliminates repeated authentication logic inside controllers

## 4. Route Layer

Routes define the application's API endpoints and map incoming HTTP requests to controller methods. Each route represents a specific functionality of the system.

### Authentication Routes

- Register User
- Login User
- User Profile

### Workout Routes

- Add Workout
- Get All Workouts
- Search Workouts
- Get Workout by ID
- Update Workout
- Delete Workout

### AI Routes

- Workout Recommendation
- Fitness Insights

## 5. Controller Layer

Controllers contain the application's business logic. They receive requests from routes, validate input data, invoke services or database operations, and generate standardized JSON responses.

### Authentication Controller

Responsible for:

- User Registration
- User Login
- User Profile Retrieval

### Workout Controller

Responsible for:

- Creating Workout Records
- Retrieving Workout Records
- Updating Workouts
- Deleting Workouts
- Searching Workouts

### AI Controller

Responsible for:

- Sending prompts to Google Gemini
- Receiving AI-generated recommendations
- Returning AI responses to clients

## 6. Service Layer

The Service Layer contains reusable business logic that can be shared among multiple controllers.

### Gemini AI Service

Responsible for:

- Preparing prompts
- Sending prompts to Google Gemini
- Receiving AI-generated responses
- Returning formatted recommendations

### JWT Service

Responsible for:

- Creating JWT Tokens
- Configuring Token Expiry
- Verifying User Identity

### Password Service

Responsible for:

- Hashing Passwords
- Comparing Passwords During Login

## 7. Database Layer

MongoDB stores all persistent application data. Mongoose acts as an Object Data Modeling (ODM) library that simplifies communication between Node.js and MongoDB.

### Collections

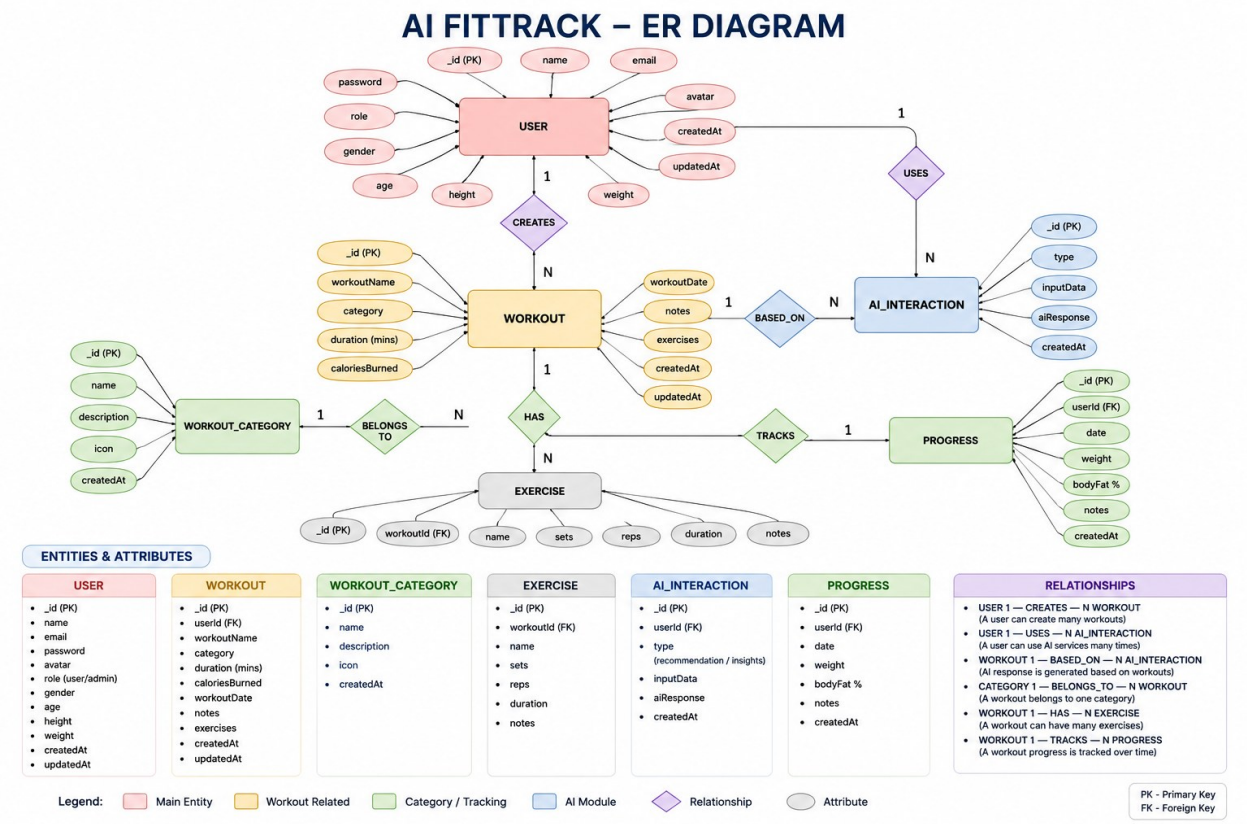
- Users Collection
- Workouts Collection

### Advantages

- Flexible Schema
- High Performance
- Easy Data Retrieval
- Scalable Architecture

# Entity Relationship (ER) Diagram Description

The AI FitTrack backend consists of two primary collections that are logically connected through a one-to-many relationship.




## 1. User Entity

The User entity stores authentication and profile information.

### Attributes

| Field | Type | Description |
|---|---|---|
| _id | ObjectId | Primary Key |
| name | String | User Name |
| email | String | Unique Email Address |
| password | String | Encrypted Password |
| createdAt | Date | Record Creation Time |
| updatedAt | Date | Last Updated Time |

### Constraints

- Email must be unique.
- Password is encrypted using bcrypt.
- Name is mandatory.
- JWT token is generated after successful login.

## 2. Workout Entity

The Workout entity stores workout-related information for each authenticated user.

### Attributes

| Field | Type | Description |
|---|---|---|
| _id | ObjectId | Primary Key |
| user | ObjectId | References User Collection |
| workoutName | String | Workout Name |
| category | String | Workout Category |

| | | |
|---|---|---|
| duration | Number | Duration (Minutes) |
| caloriesBurned | Number | Calories Burned |
| workoutDate | Date | Workout Date |
| createdAt | Date | Created Timestamp |
| updatedAt | Date | Updated Timestamp |

### `Relationship`

### User → Workout

**Relationship Type:** One-to-Many

A single user can create multiple workout records. Each workout belongs to only one authenticated user.

Example:

John Doe

- Morning Running
- Evening Jogging
- Upper Body Strength
- Cycling Session

This relationship is maintained using the **user** field inside the Workout schema, which stores the ObjectId of the corresponding user.

# Key Features

The FitTrack AI  provides a complete backend solution for workout tracking and AI-assisted fitness guidance.

### `1. Secure Authentication`

The application implements JWT-based authentication.

Features include:

- User Registration
- Secure Login
- Password Encryption
- JWT Generation
- Protected APIs

## 2. Workout Management

Users can perform complete CRUD operations.

Supported Operations

- Add Workout
- View Workout
- Update Workout
- Delete Workout

Each workout is linked to its authenticated user.

## 3. Workout Search

Users can search workouts using:

- Workout Name
- Workout Category
- Workout Date

This improves accessibility of workout history.

## 4. Google Gemini Integration

The backend communicates with Google Gemini AI to generate intelligent workout recommendations.

Input

- Age
- Fitness Goal
- Experience Level

Output

- Weekly Workout Plan

- Suggested Exercises
- Training Tips
- Safety Recommendations

## 5. AI Fitness Insights

The AI module analyzes workout statistics.

Input

- Total Workouts
- Average Duration
- Calories Burned

Output

- Performance Analysis
- Improvement Suggestions
- Motivational Advice
- Fitness Progress Summary

## 6. Centralized Error Handling

The application returns meaningful JSON error responses for:

- Invalid Credentials
- Missing Authentication Token
- Validation Errors
- Database Errors
- Internal Server Errors

## 7. RESTful API Design

The backend follows REST principles.

Supported HTTP Methods

- GET
- POST
- PUT
- DELETE

Responses are returned in JSON format.

### 8. Modular Architecture

The project separates responsibilities into

- Models
- Controllers
- Routes
- Middleware
- Services
- Configuration

This improves maintainability and scalability.

# Roles and Responsibilities

The AI FitTrack backend supports one primary role.

### Authenticated User

### Responsibilities

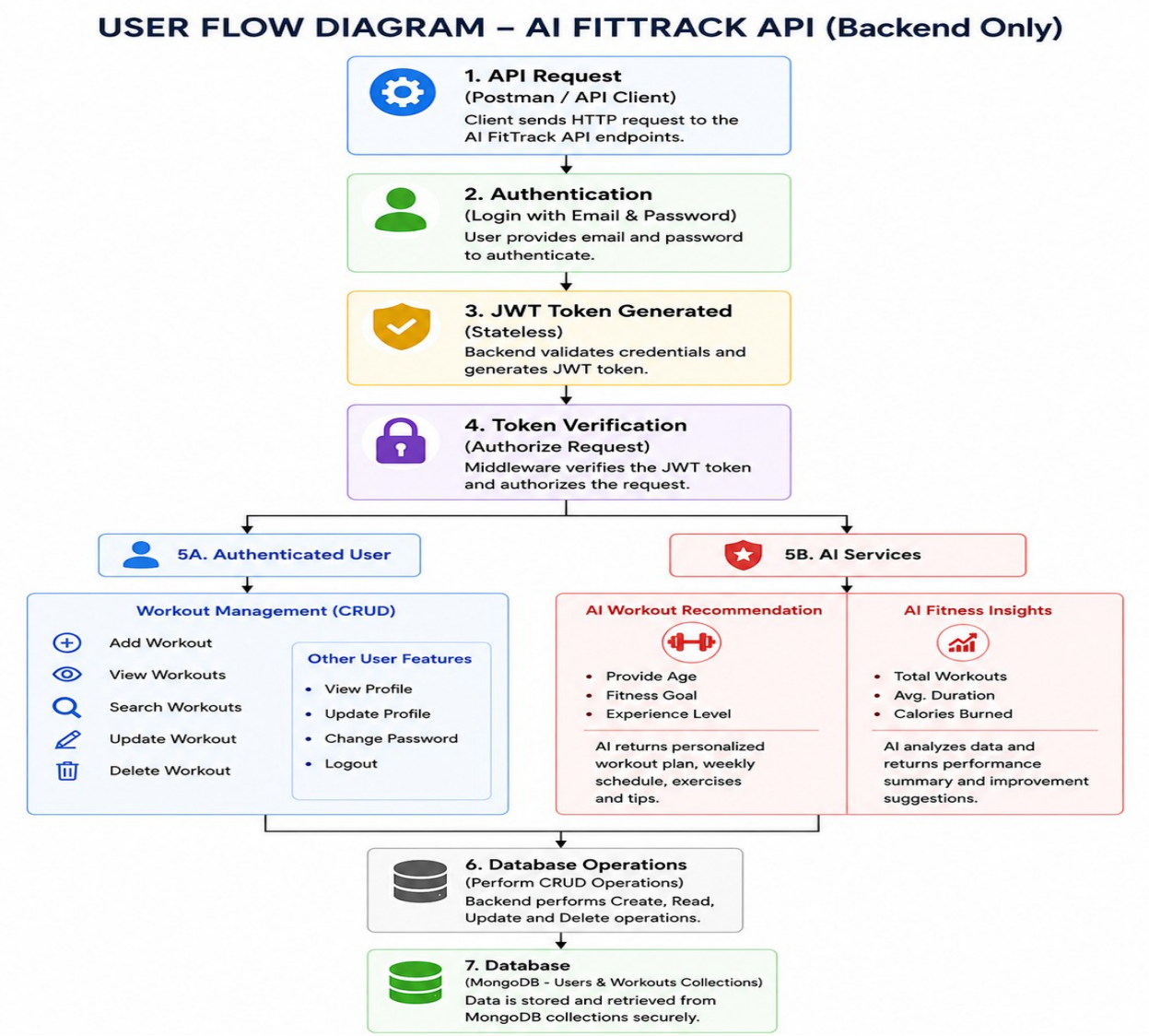
- Register an account.
- Login securely.
- View profile information.
- Add workout records.
- View workout history.
- Search workouts.
- Update workouts.
- Delete workouts.
- Request AI workout recommendations.
- Request AI fitness insights.

### Permissions

| Operation | Permission |
|---|---|
| Register | Yes |
| Login | Yes |

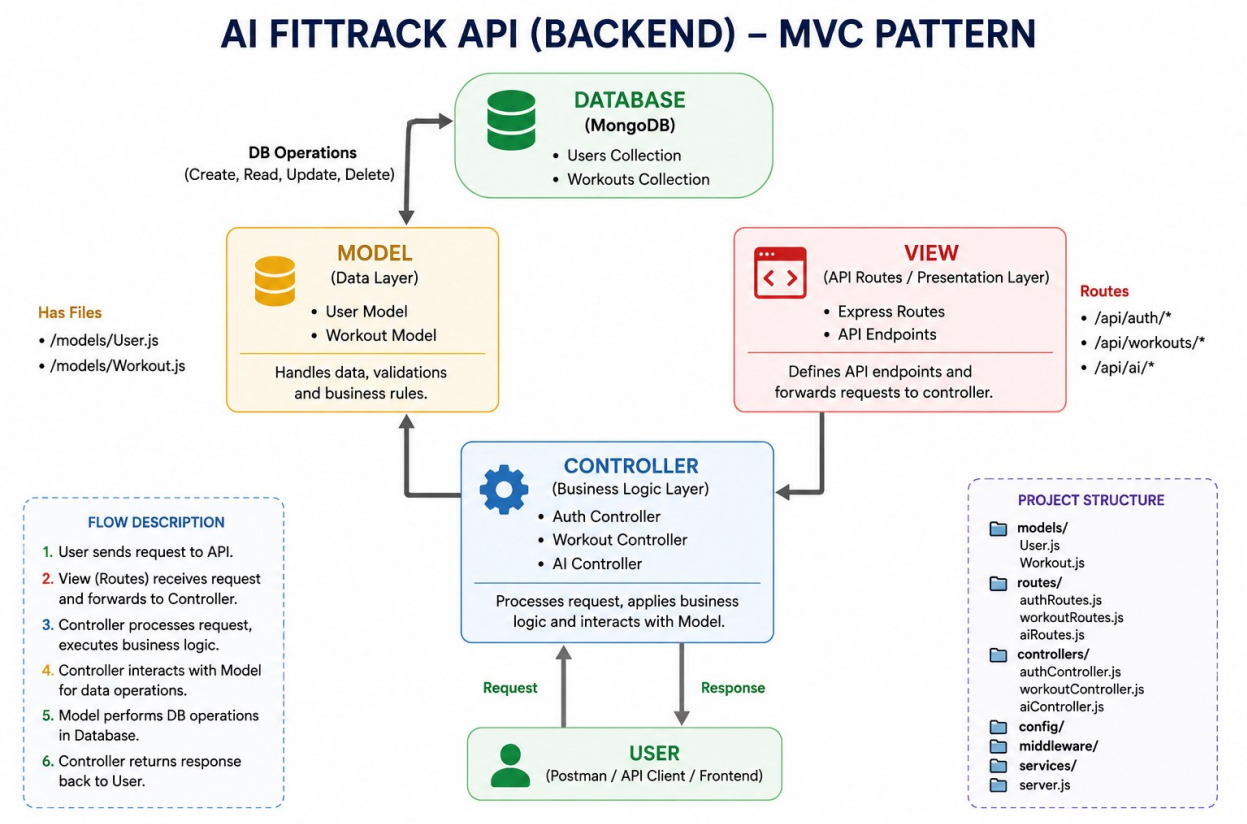
| | |
|---|---|
| View Profile | Yes |
| Add Workout | Yes |
| Get Workouts | Yes |
| Search Workouts | Yes |
| Update Workout | Yes |
| Delete Workout | Yes |
| AI Recommendation | Yes |
| Fitness Insights | Yes |

# User Flow




# MVC Architecture Pattern

The FitTrack AI  is designed using the **Model–View–Controller (MVC)** architecture pattern. This architecture separates application logic into independent components, improving code organization, maintainability, and scalability.




## Model Layer (Data Layer)

The Model Layer defines the structure of the application's data using Mongoose schemas. It communicates directly with MongoDB and performs database operations.

Models in the project include:

- User Model
- Workout Model

Responsibilities:

- Schema Definition
- Data Validation
- CRUD Operations
- Collection Mapping

## View Layer (API Interface)

Since AI FitTrack is a backend REST API, there are no traditional UI templates. The View Layer is represented by API consumers such as React applications, mobile applications, or Postman, which interact with the backend through HTTP requests and receive JSON responses.

Responsibilities:

- Send Requests
- Receive JSON Responses
- Display Data
- Handle Client-side Interaction

### Controller Layer (Business Logic)

The Controller Layer acts as the intermediary between the View and Model. It processes incoming requests, validates input, invokes services and models, and sends structured JSON responses back to the client.

Responsibilities:

- Request Validation
- Business Logic Execution
- Database Interaction
- AI Service Invocation
- Response Generation

The separation of these three layers ensures loose coupling, simplifies testing, enhances code readability, and makes the application easier to extend with future features.

# Epic 2: Project Setup and Configuration

The FitTrack AI  project follows a well-structured backend architecture that separates business logic, routing, database operations, middleware, and AI services into individual modules. This modular design improves code readability, maintainability, and scalability while making the project easier to debug and extend.

The backend is developed using **Node.js** and **Express.js**, with **MongoDB** as the primary database and **Google Gemini AI** for intelligent fitness recommendations.

# Creating Project Folder

### Step 1: Create the Project Folder

Create a new folder with the project name.

```
AI-FitTrack-API
```

## Step 2: Open the Project

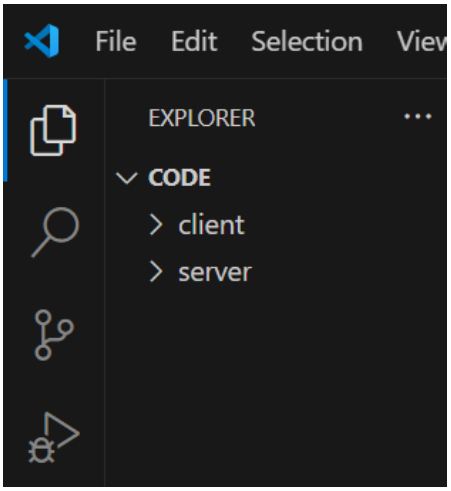
Open the project folder in **Visual Studio Code**.

## Step 3: Create the Server Folder

Inside the project directory, create a folder named:

```
server
```

All backend source code will be maintained inside this folder.




## Step 4: Initialize Node Project

Open the terminal inside the **server** folder and execute:

```
npm init -y
```

This command generates the **package.json** file containing project metadata and dependency information.

## Step 5: Install Required Packages

Install the required dependencies.

```
npm install express mongoose dotenv bcryptjs jsonwebtoken cors
```

Install Nodemon as a development dependency.

```
npm install --save-dev nodemon
```

### Environment Variables

Sensitive information is stored in the **.env** file.

Example

```
PORT=5000

MONGO_URI=mongodb://localhost:27017/aifittrack

JWT_SECRET=your_secret_key

GEMINI_API_KEY=your_google_gemini_api_key
```

Using environment variables improves security by preventing sensitive information from being exposed in source code.

# Epic 3: Backend Development

Backend development focuses on implementing REST APIs that allow authenticated users to manage workouts and interact with Google Gemini AI.

The project is divided into multiple modules that collectively perform authentication, workout management, AI integration, and database operations.

Major modules include:

- Server Configuration
- Database Configuration
- Authentication
- Workout Management
- AI Recommendation
- Middleware
- Error Handling

**Video Link:**
https://drive.google.com/file/d/1KswrctSaPl0eOYyJ3GkbbVHvHsU3EfZJ/view?usp=sharing

# Epic 4: Database Configuration

The FitTrack AI  uses **MongoDB** as its primary NoSQL database for storing user credentials and workout information. **Mongoose** serves as the Object Data Modeling (ODM) library, enabling seamless interaction between the application and the MongoDB database. Mongoose provides schema validation, middleware support, data relationships, and simplified CRUD operations.

The database layer is responsible for maintaining data integrity, validating incoming requests, and ensuring secure storage of user information.

# MongoDB Configuration

MongoDB stores application data in collections rather than traditional relational database tables. The FitTrack AI  maintains separate collections for users and workouts.

### Database Collections

- Users Collection
- Workouts Collection

Each collection stores JSON-like documents (BSON format), allowing flexible schema design while maintaining efficient data retrieval.

### Installing Mongoose

Install Mongoose using npm.

```
npm install mongoose
```

Mongoose acts as the bridge between the Node.js application and MongoDB by providing schema definitions, validation, and database methods.

### MongoDB Connection

The database connection is configured inside the **config/db.js** file. This module establishes a connection to the MongoDB server when the application starts.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

### Responsibilities

- Connect to MongoDB
- Handle Connection Errors
- Display Connection Status
- Export Database Connection

### Advantages

- Centralized database configuration
- Reusable connection module
- Automatic connection management
- Simplified error handling

# Mongoose Schemas

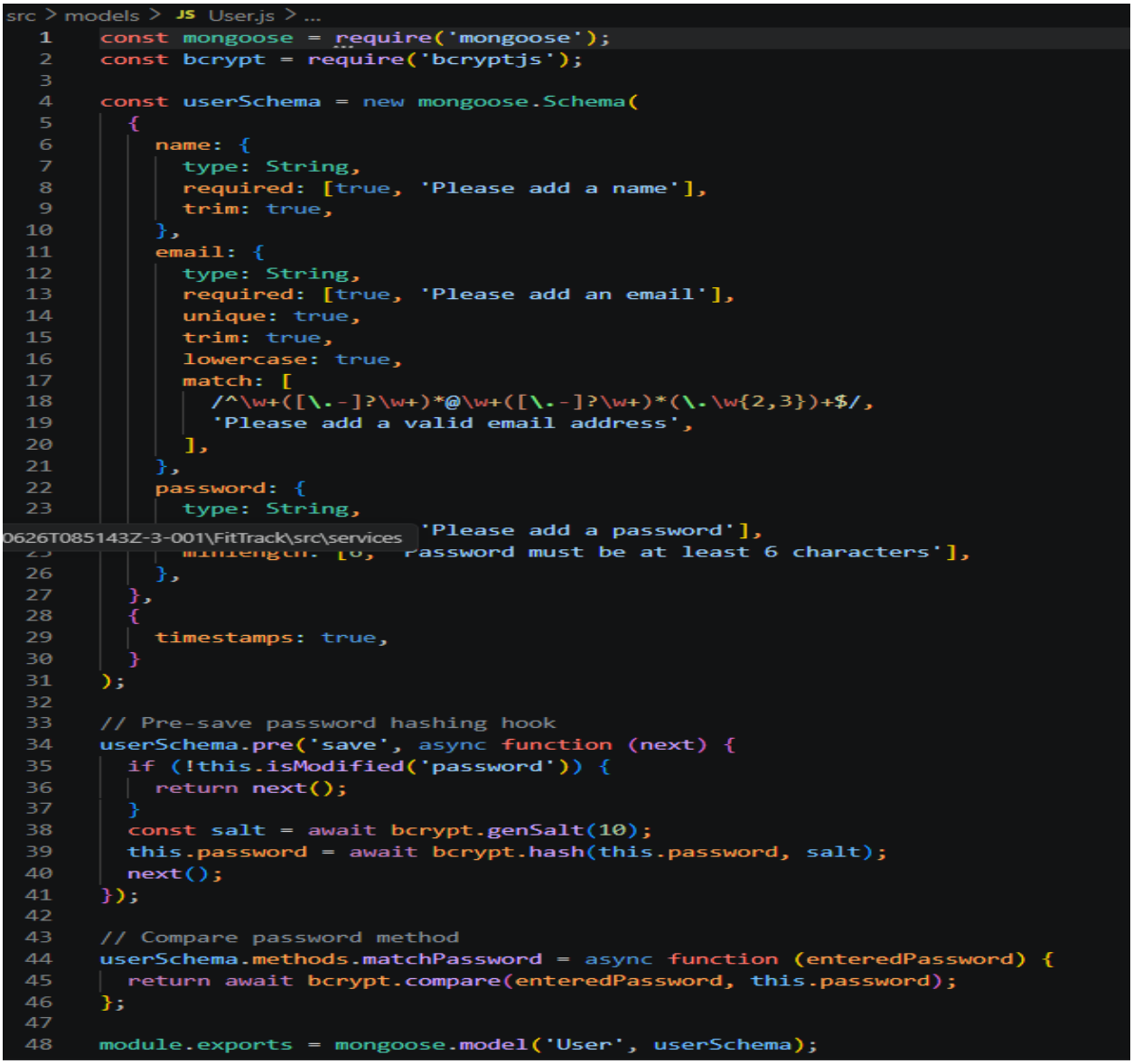
The AI FitTrack backend contains two primary Mongoose schemas:

1. User Schema
2. Workout Schema

Schemas define the structure, validation rules, and relationships for MongoDB documents.

## 1. User Schema (models/User.js)

The User schema stores authentication information for registered users.




### Validation Rules

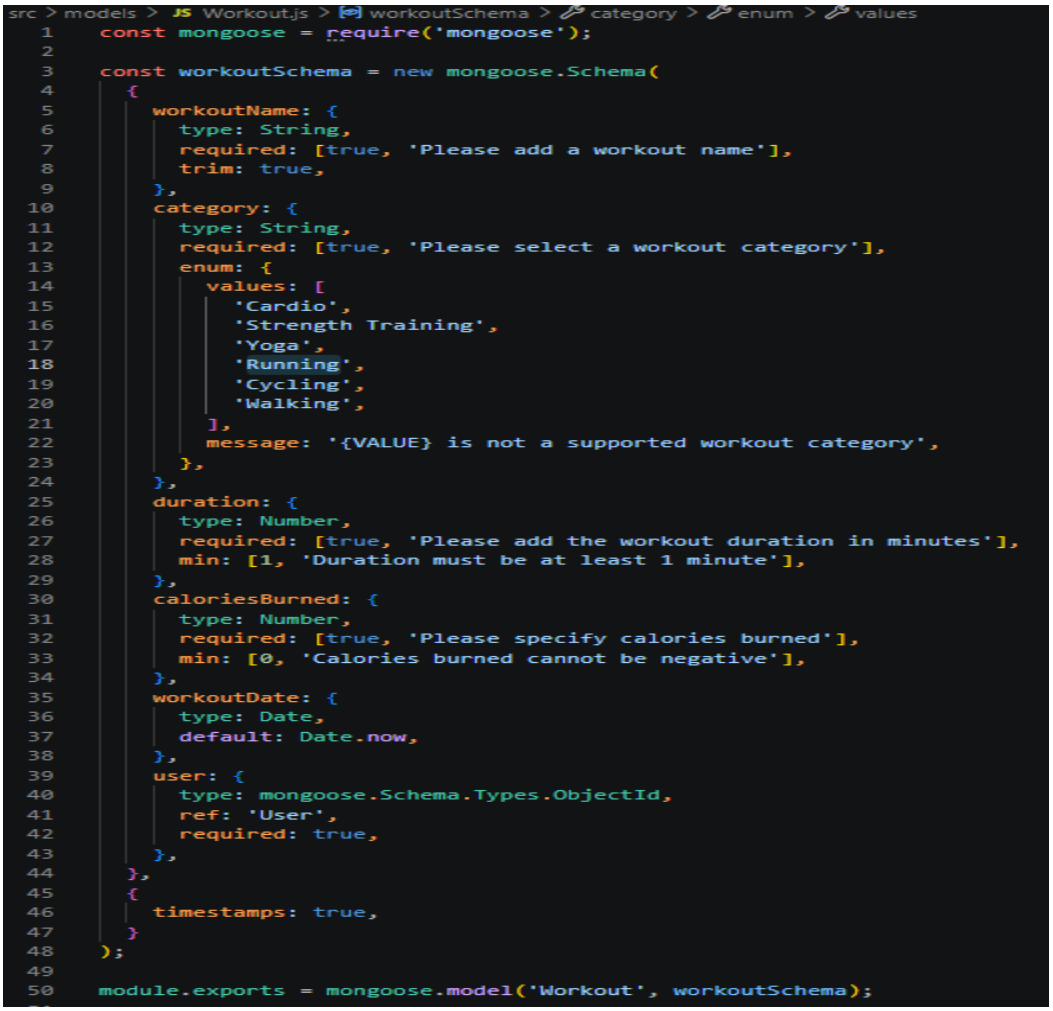
- Name cannot be empty.
- Email must be unique.
- Password is mandatory.
- Password is encrypted using bcrypt.js before saving.
- Automatic timestamps are enabled.

### User Schema Features

- Email uniqueness validation
- Password hashing
- Automatic timestamps
- Secure authentication support
- JWT integration

## 2. Workout Schema (models/Workout.js)

The Workout schema stores workout information created by authenticated users.




### Validation Rules

- Workout Name is mandatory.
- Category is mandatory.
- Duration must be greater than zero.
- Calories Burned must be greater than or equal to zero.
- Workout Date is required.

- Every workout belongs to one authenticated user.

### Workout Schema Features

- Schema validation
- Automatic timestamps
- User relationship
- Easy search support
- Efficient CRUD operation

# Epic 5: Project Execution

· Install Postman or ThunderClient (VS Code extension).

· Start the backend server:

## “npm start”

### API Testing

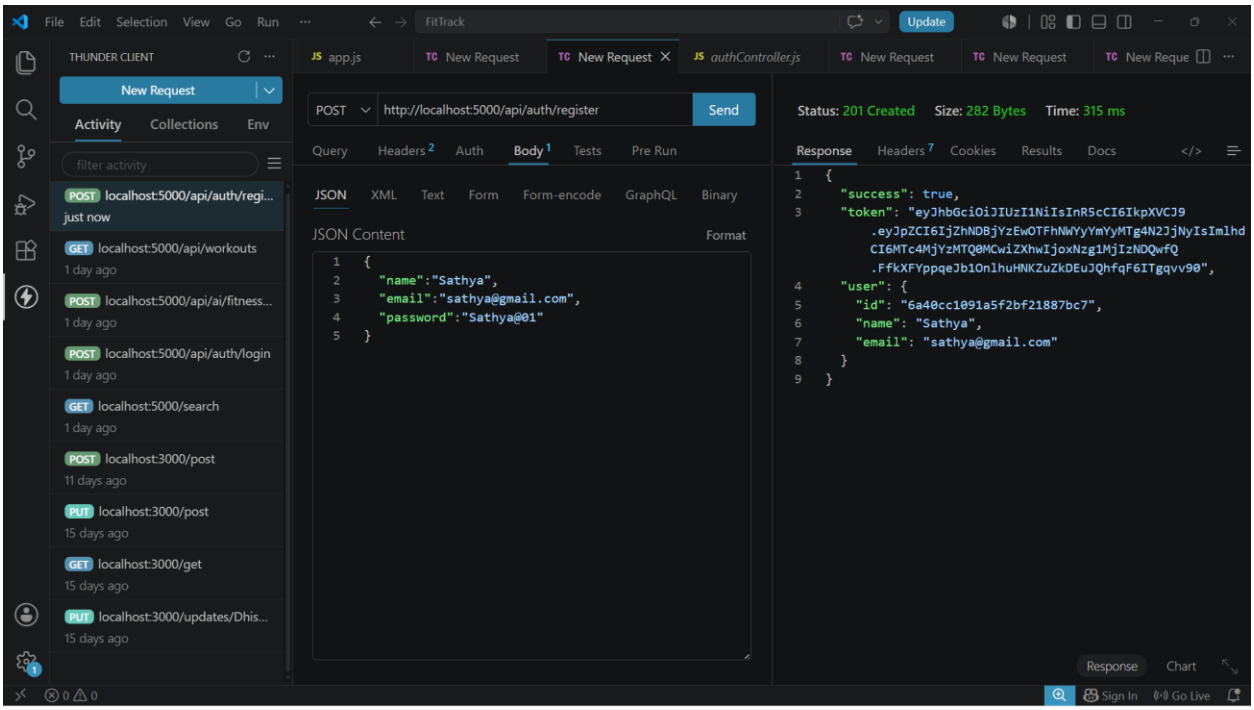
API testing ensures that every endpoint functions correctly and returns the expected response. The FitTrack AI  is tested using **Postman**, allowing developers to verify request handling, authentication, database operations, and AI integration.

Testing is performed in the following order:

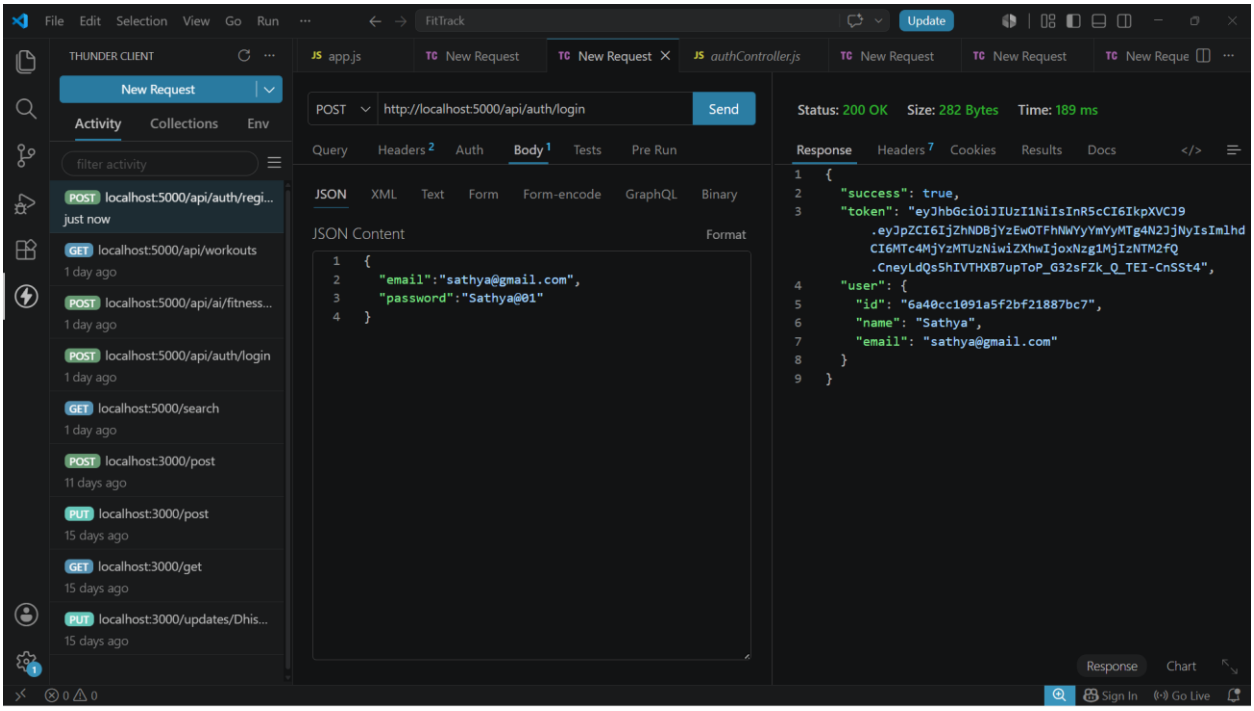
1. Authentication APIs
2. Workout APIs
3. AI APIs

# Authentication API Testing

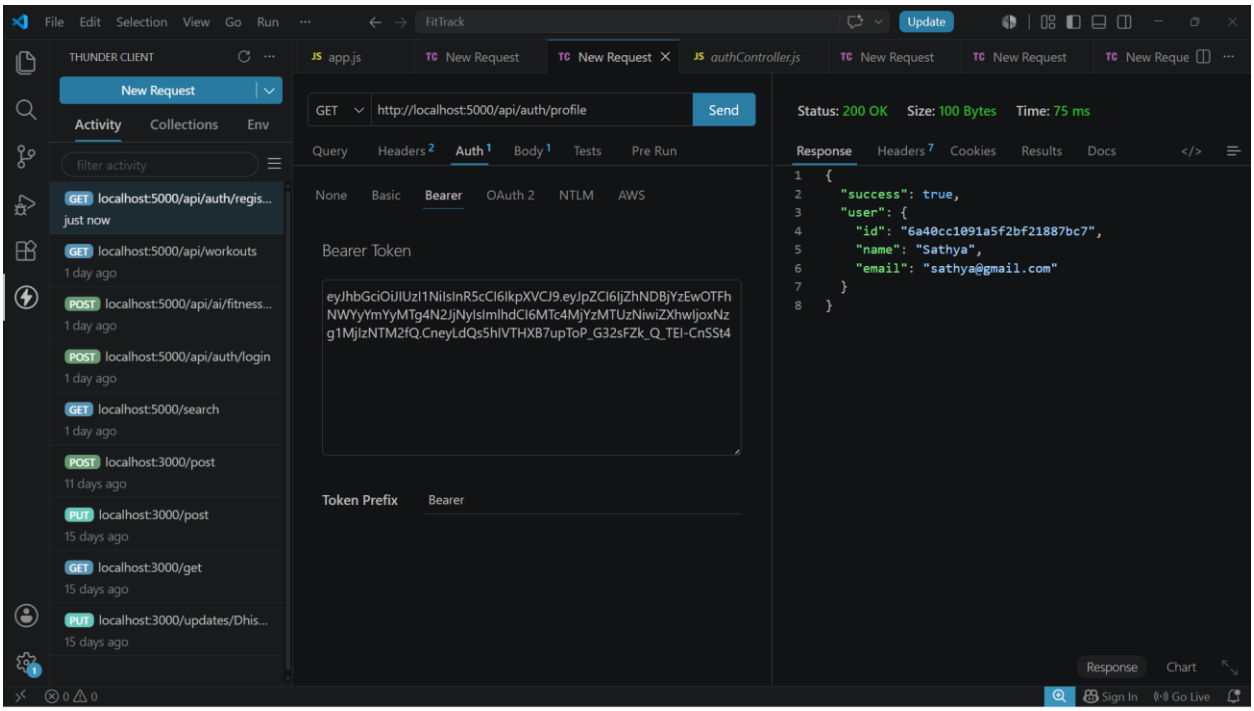
## Register User




## Login User

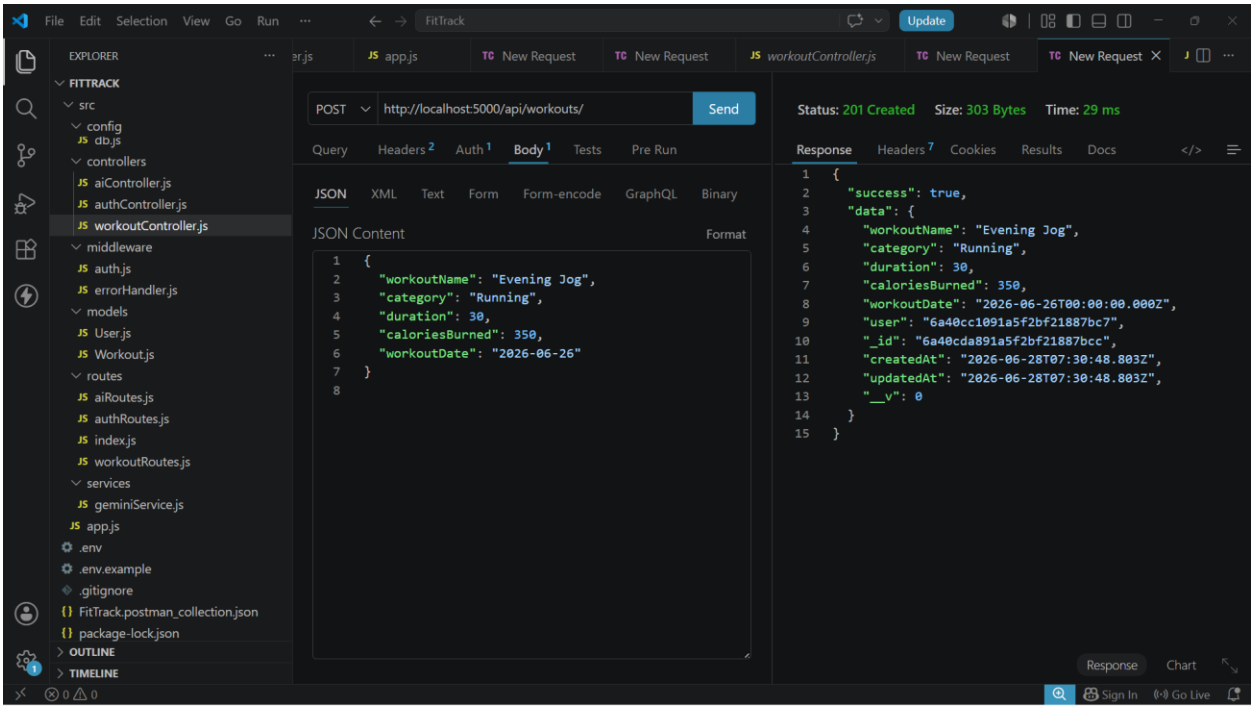



## Get Profile

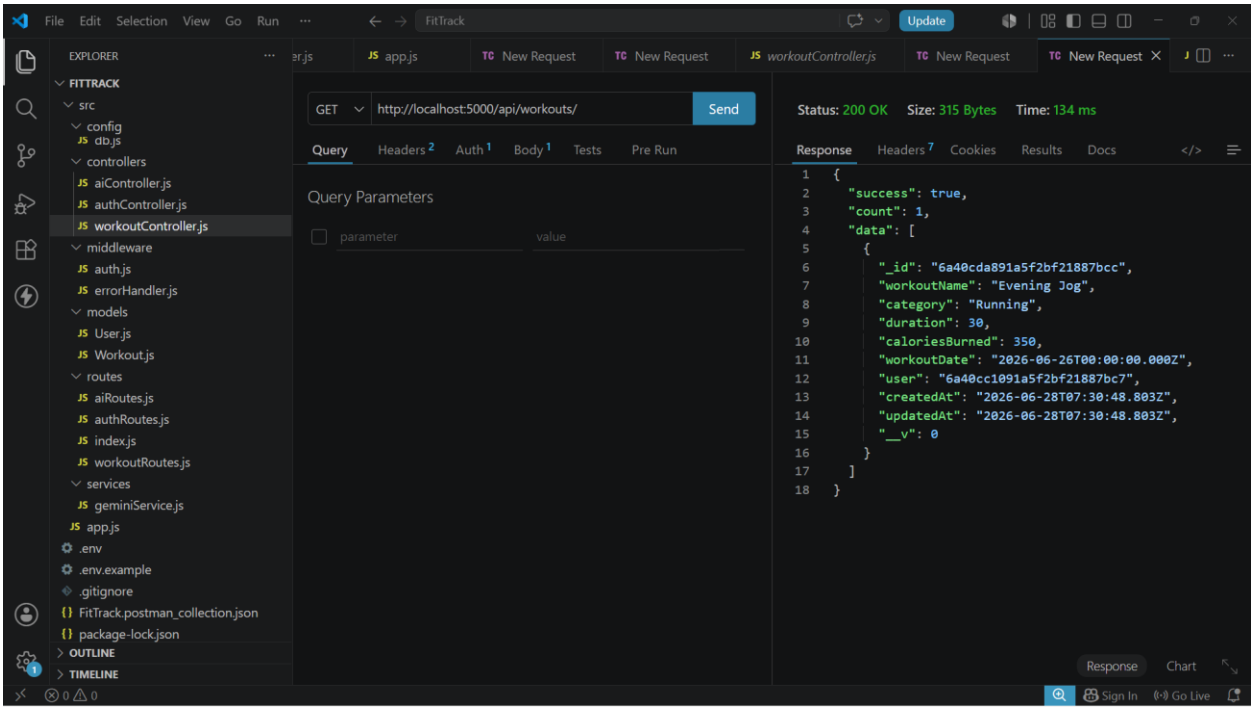



# Workout API Testing

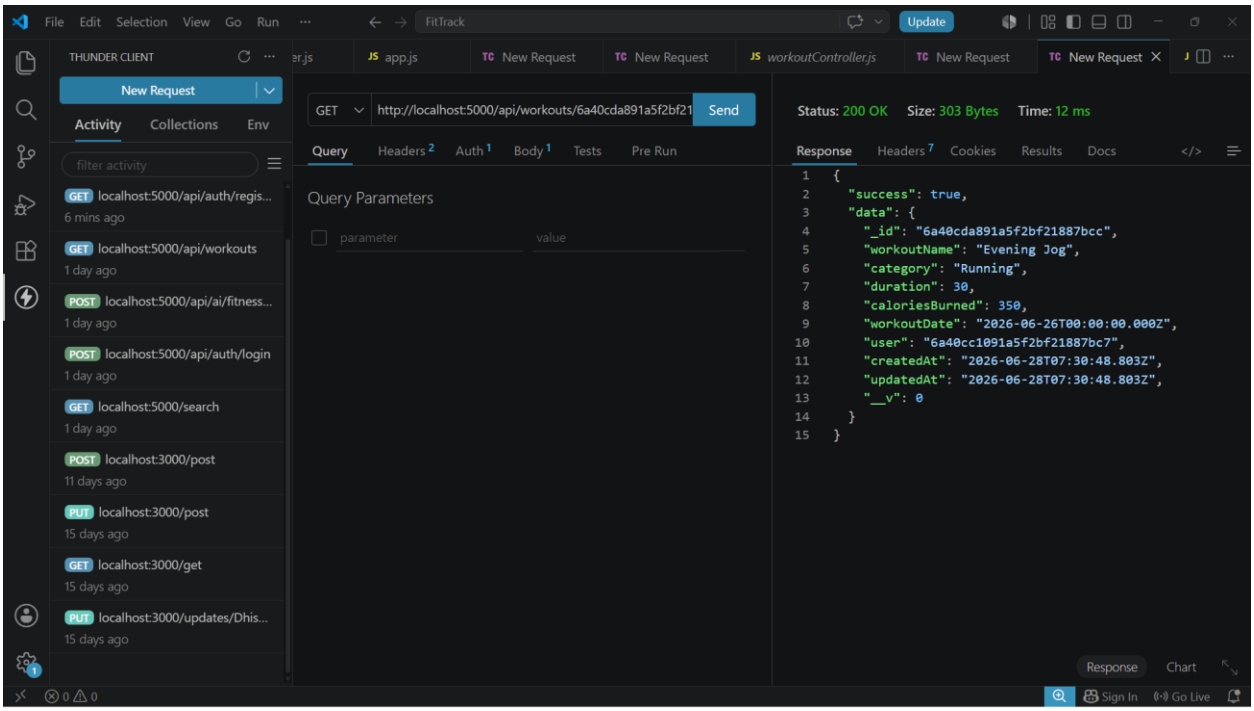
## Add Workout



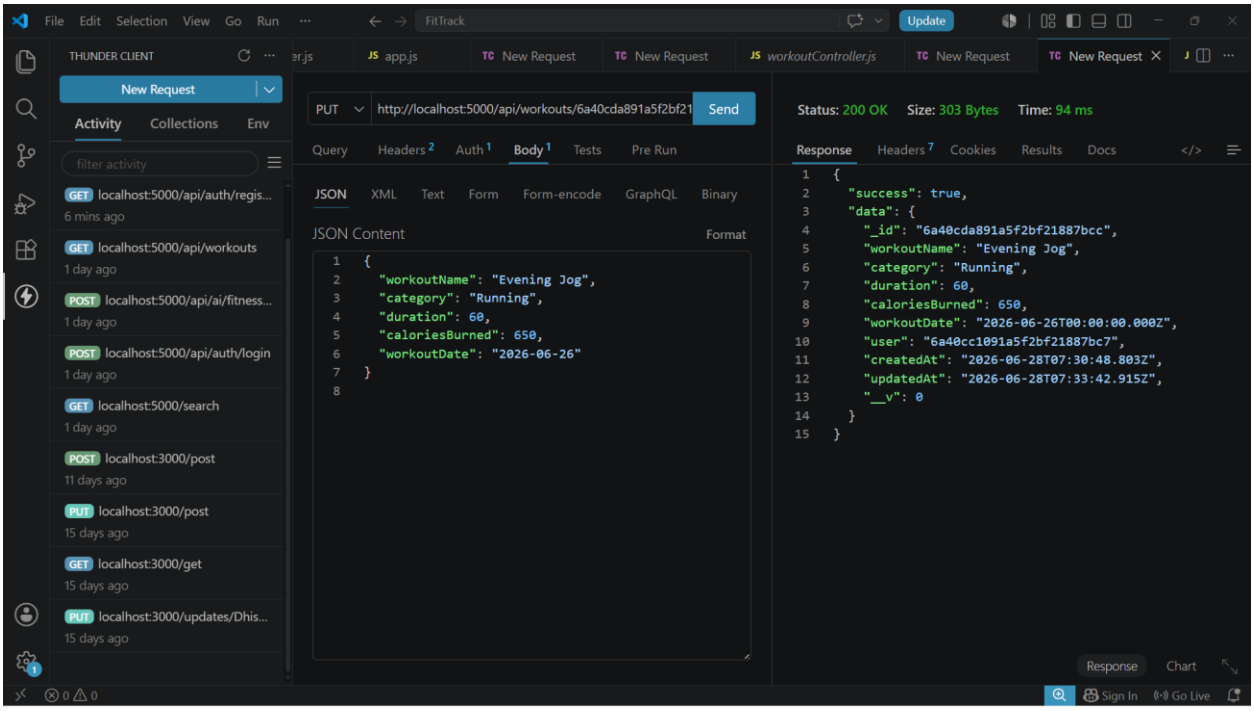

## Get All Workouts



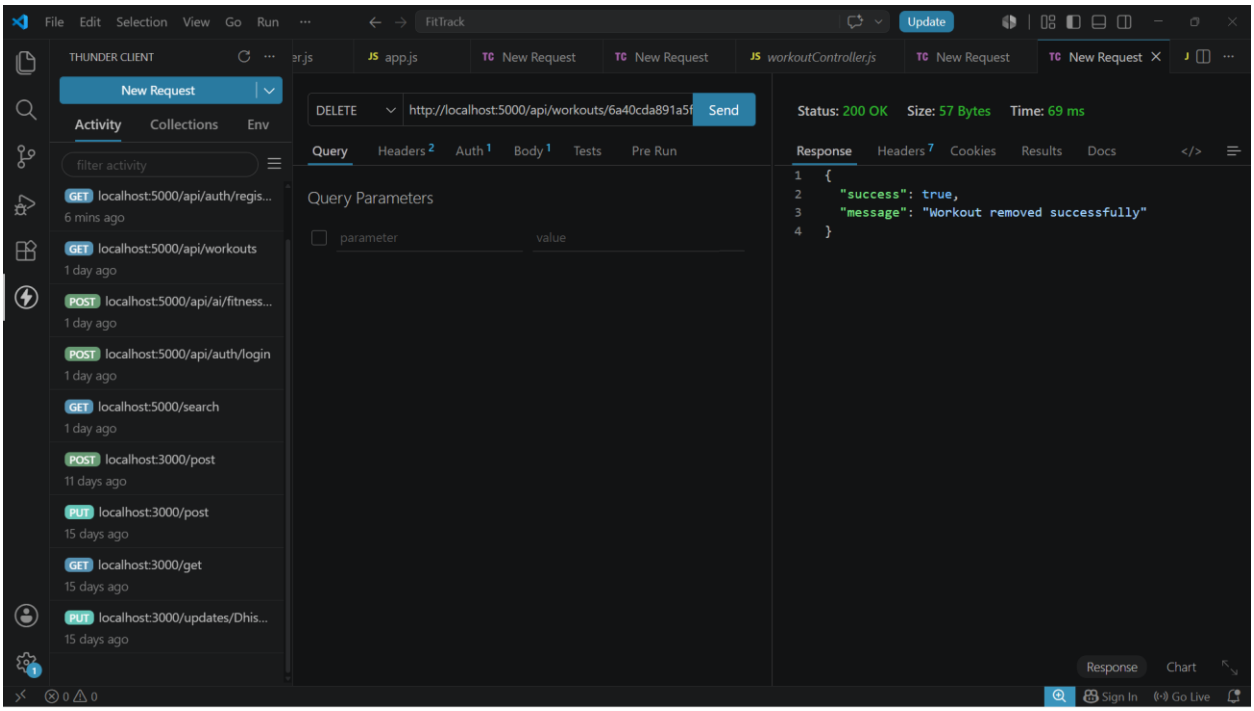

## Get Workout By ID




## Update Workout

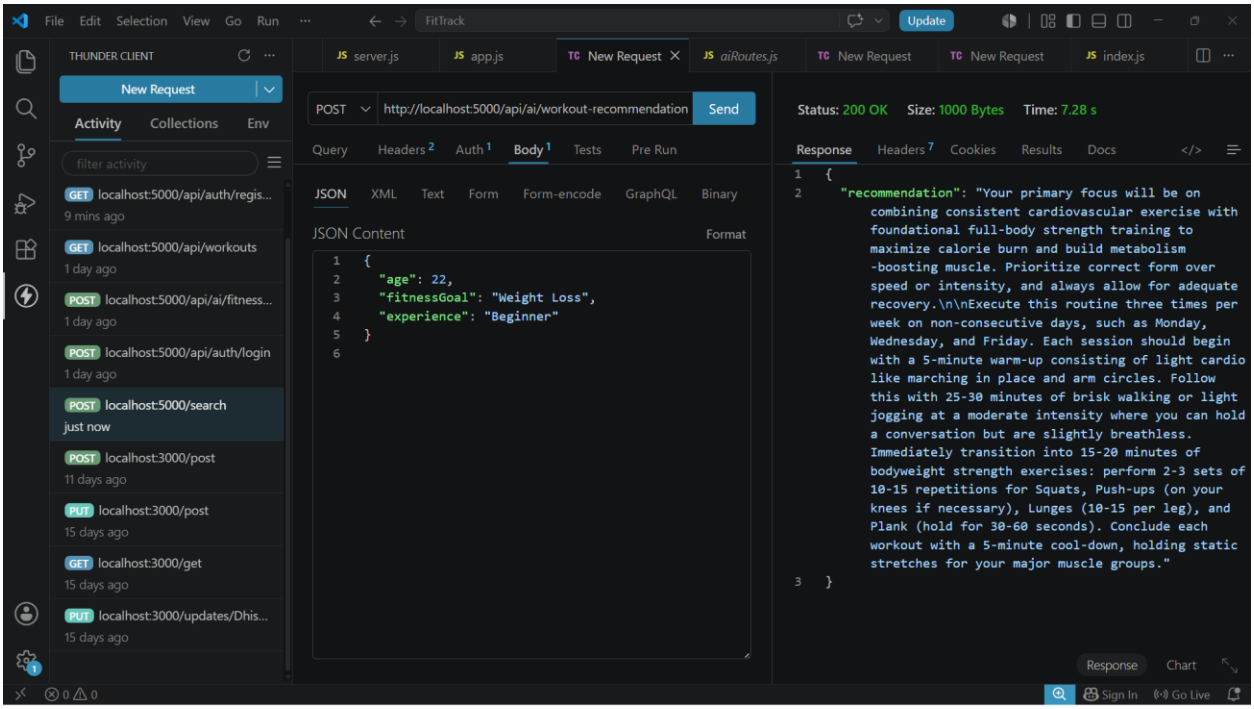



## Delete Workout

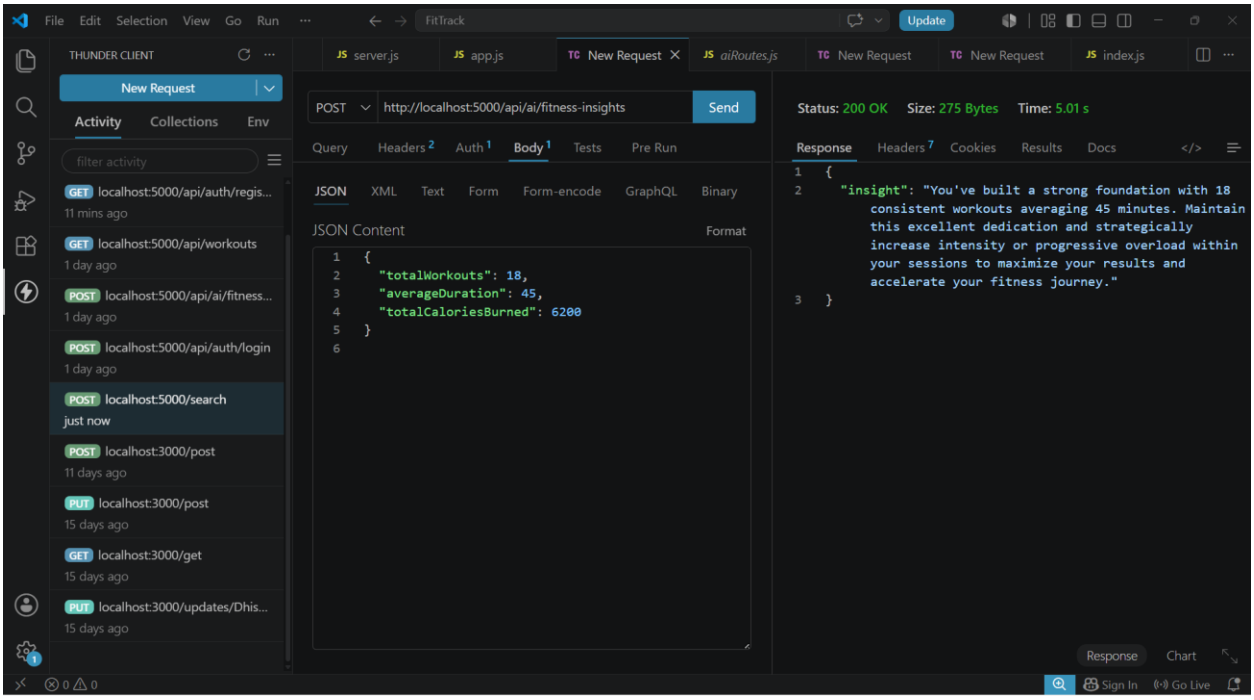



# AI API Testing

## Workout Recommendation




## Fitness Insights




**Demo Video Link:**
https://drive.google.com/file/d/1KswrctSaPI0eOYyJ3GkbbVHvHsU3EfZJ/view?usp=drive_link

**API TESTING VIDEO : https://drive.google.com/file/d/1TCXG48a-78ryxF3gipdRMwRBwuhAgPaa/view?usp=drive_link**

**CodeDrive:https://drive.google.com/drive/folders/1hwmKxkHQkUI62JR1svI3xCQ2ShZ6Lj9y.**